SCIPY 2026 · PRODUCTION ML OBSERVABILITY

Right Predictions, Wrong Reasons

Explanation Drift Monitoring in Production — tracking *how* a model reasons, not just whether it is right

Avik Basu · Intuit, Inc.

Open source · Apache-2.0

`pip install shap-monitor`

github.com/ab93/shap-monitor

THE PROBLEM

A model can be right for the wrong reasons

A model can hold its headline metrics steady while the features driving its decisions change underneath it. When that happens, it has quietly become a **different model** — and the eventual failure is sudden and hard to diagnose.

The standard monitoring stack has a structural blind spot here:

- **Performance monitoring** (F1, AUC) is the ground truth — but it *lags*. Labels for loans, fraud, and churn arrive weeks to months late.
- **Input-drift monitoring** needs no labels — but only asks *"did the data change?"*, not *"did the model's use of the data change?"*

THE IDEA

A third signal: explanation drift

A statistically detectable change over time in the distribution of a model's **SHAP attributions** on production inputs.

- `P(X)` **Input drift** — did the data change?
- `P(φ(X))` **Explanation drift** — did the model's reasoning change?
- `F1 / AUC` **Performance** — are the outputs wrong yet?

$\phi(X)$ couples the *data* and the *current model*, so it reacts to changes a feature-only view misses. It needs **no labels** — a **leading** signal in delayed-label regimes — and because it is per-feature, it points at *which* part of the reasoning moved.

It can arise from a **covariate shift** that changes which features the model leans on, from **concept drift**, or from a **changed model or upstream transform** — and attributions move even on identical inputs.

WHAT WE TRACK

The distribution of attributions, over time

SHAP splits each prediction into per-feature contributions that add up exactly:

$$f(x) = \phi_0 + \sum_i \phi_i$$

We don't watch any single explanation — we watch how two summaries of the *distribution* of ϕ_i evolve:

- **Mean |SHAP|** $E[|\phi_i|]$ — global importance: *how much* a feature moves predictions.
- **Mean signed SHAP** $E[\phi_i]$ — the typical *direction* of its effect.

A change in **either** is a change in the model's reasoning.

For tree models, `TreeExplainer` computes these attributions **exactly** and efficiently — making per-prediction explanation practical at production volumes.

CASE STUDY · UCI ADULT INCOME

A stable score hiding a reasoning shift

A deliberate demographic shift: train a LightGBM classifier on the **younger** population (age ≤ 45), then serve the **older** population (age > 45) as drifted production traffic. Exact attributions via `TreeExplainer`; all drift stats via `shap-monitor`.

F1 SCORE (ACCURACY)	EXPLANATION-SHIFT AUC
0.71 ⟶ 0.72	0.50 ⟶ 0.97

Watching F1 alone — which in production we would not even have for weeks — we would have seen **nothing worth investigating**.

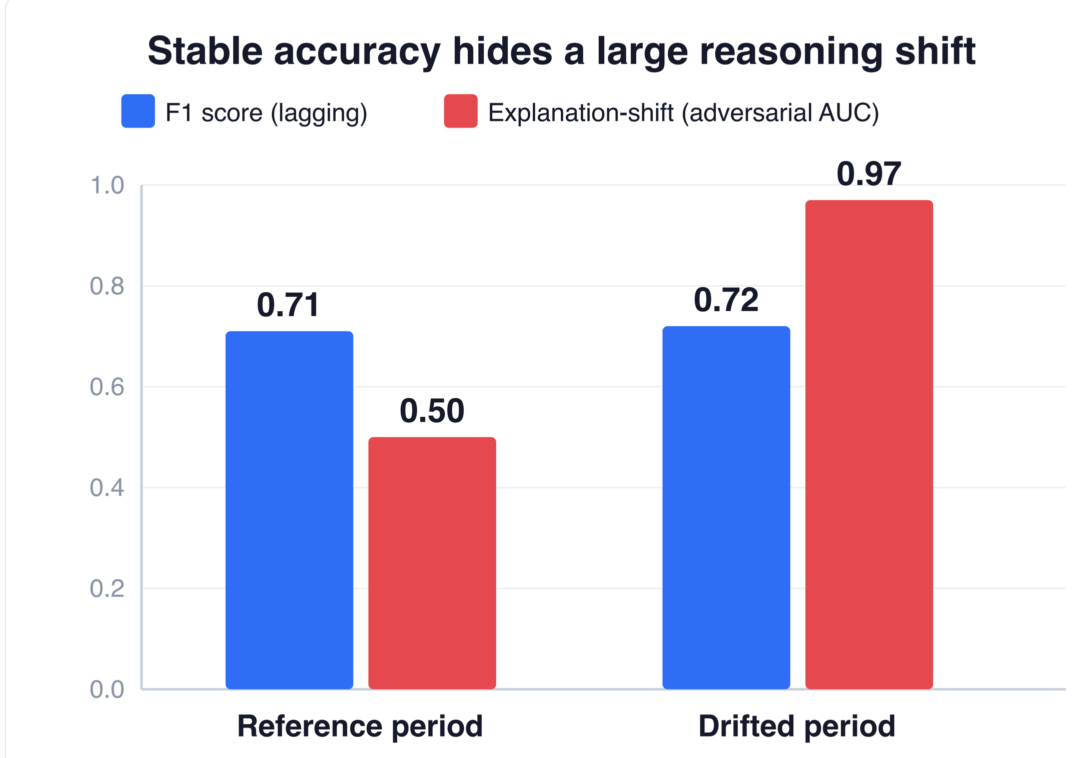

Fig. 1 — The blind spot. F1 is flat across the shift (0.71→0.72), so performance monitoring sees nothing. The adversarial-validation score over the SHAP attributions rises from 0.50 (no-drift control) to **0.97** — the reasoning in the two regimes is almost perfectly separable.

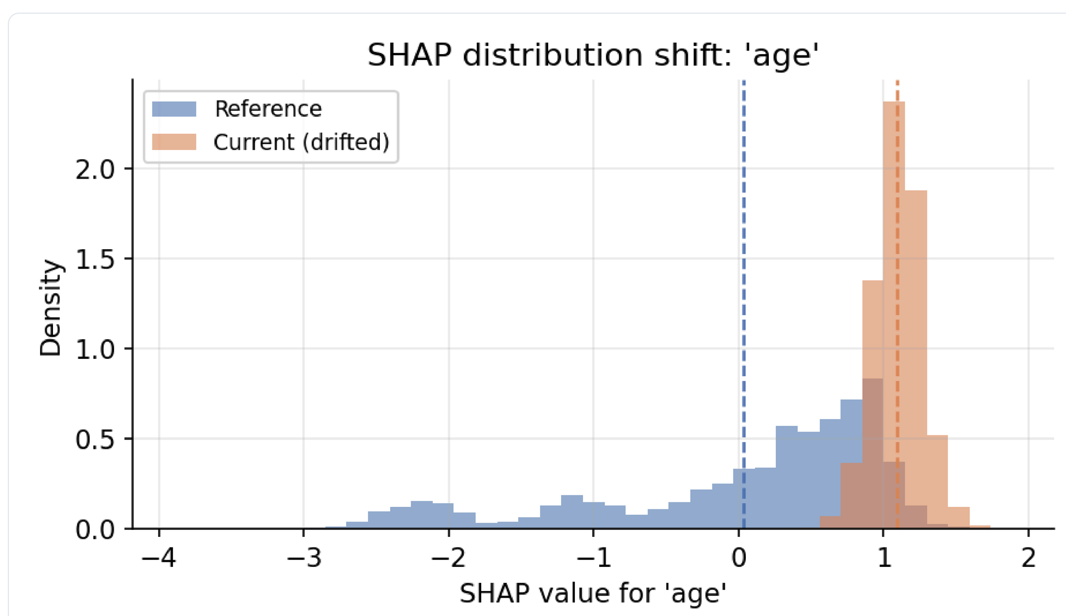

Fig. 3 — Inside the top-drifted feature. The **age** SHAP distribution goes from broad and bimodal (reference, blue) to a tight, strongly positive cluster (drifted, orange). For the older population, `age` has become a consistent, large *positive* driver of the high-income prediction — a qualitative change in the model's reasoning that accuracy never reveals.

A flat headline metric is not evidence a model is behaving as it did at validation time. Explanation drift makes the difference observable — without waiting for labels.

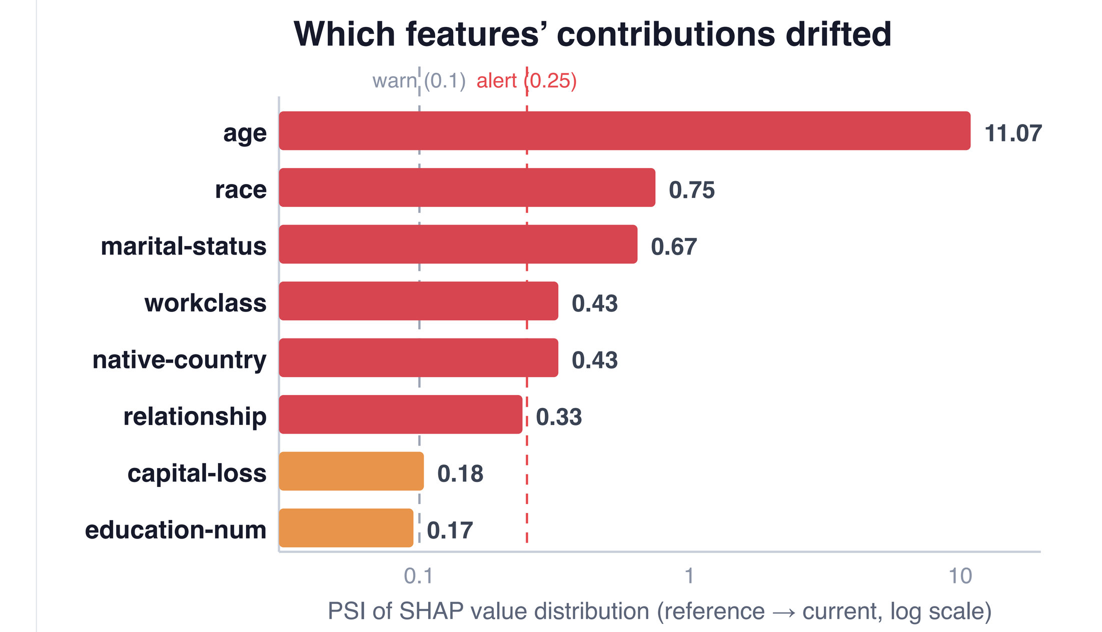

Fig. 2 — PSI localizes the shift. The **age** attribution distribution moves by a PSI of **11.07**, far past the 0.25 alert line, and five other features also cross it. The model reorganized *which* features drive its decisions.

WHAT SHAP-MONITOR FLAGGED

Beyond the figures

On the very same traffic where F1 held steady, the analyzer also surfaced:

- **+33%** mean |SHAP| on `age`, **+82%** on `workclass` — features growing sharply more influential.
- **Sign flips** on `marital-status` & `workclass` — their typical contribution *reversed direction*.

A sign flip on an important feature means a relationship learned at training time **no longer holds**. None of it is visible in accuracy.

DETECTION SIGNALS

Three complementary read-outs

Per-feature PSI — *how much?*

Population Stability Index on each feature's SHAP distribution. Cheap, interpretable, localizes drift. Bands: <0.1 stable · 0.1–0.25 warn · >0.25 alert.

Adversarial AUC — *distinguishable at all?*

Train a classifier to separate reference vs. current attribution vectors. AUC ≈ 0.5 = no drift; → 1.0 = joint reasoning easily separable. Catches multivariate shifts PSI misses.

Magnitude / rank / sign — *in what way?*

Change in mean |SHAP|, importance rank, and direction. A **sign flip** on an important feature is the highest-signal warning of all.

FAILURE PATTERNS IT CATCHES

Concept drift

Same inputs, shifted feature→target relationships. PSI on affected SHAP rises before labels confirm the damage.

Model-version regression

A new version matches the old one's accuracy but reasons differently (e.g. leans on a leaky proxy). High adversarial AUC between versions on the same traffic.

Silent data-pipeline errors

A feature pinned to a plausible constant stays in-range for input monitors, but the model's *use* of it collapses — and attribution catches it.

THE TOOL · SHAP-MONITOR

log_batch (sample traffic) → **SHAP** (attributions) → **Parquet** (date=YYYY-MM-DD/) → **Analyzer** (PSI · AUC)

```python
from shapmonitor import SHAPMonitor
monitor = SHAPMonitor(
    explainer=shap.TreeExplainer(model),
    sample_rate=0.05, # explain 5% of traffic
    data_dir="/var/log/shap")
monitor.log_batch(X, predictions)   # log inline

drift = analyzer.compare_time_periods(ref, curr)
drift[drift["psi"] ≥ 0.25]   # significant shifts
```

```
(shap-monitor-py3.13) → shap-monitor git:(feat/cli) ✗ shapmonitor report drift \
--data-dir demo/shap_logs --ref last-14d..last-7d --curr last-7d..now
Drift Report (ref: 2026-04-04 00:00:00→2026-04-11 00:00:00 vs curr: 2026-04-11
0 alerts · 2 warnings · 6 stable
```

Feature	PSI	Status	|SHAP| ref	|SHAP| curr	Δ |SHAP|	Rank Δ
payment_history	0.2356	warning	0.0406	0.0539	0.0134	—
credit_score	0.1010	warning	0.0580	0.0553	-0.0027	—
employment_years	0.0832	stable	0.0791	0.0910	0.0118	—
num_accounts	0.0498	stable	0.0059	0.0043	-0.0016	—
debt_ratio	0.0414	stable	0.0343	0.0336	-0.0006	—
recent_inquiries	0.0294	stable	0.1041	0.0990	-0.0051	—
age	0.0205	stable	0.1154	0.1178	0.0024	—
income	0.0085	stable	0.0127	0.0130	0.0003	—

`shapmonitor report drift` — cron-friendly CLI, `--json` pipes into any alerting rule.

IN PRODUCTION

Storage, alerting & overhead

Storage: attributions written as **Parquet**, Hive-partitioned by date (`date=YYYY-MM-DD/`) so time-range queries prune to the relevant partitions. Each record = timestamp · batch UUID · model version · per-feature SHAP. Readable by pandas, DuckDB, Spark; the backend is a swappable protocol.

Alerting: a threshold policy — page when any *important* feature's **PSI ≥ 0.25**, or the **adversarial AUC** vs. a fixed baseline exceeds ~0.8. Gate on a minimum mean |SHAP| to avoid noisy alarms.

Overhead: the dominant cost is computing explanations. Explain a **sampled fraction** inline (trees) or out-of-band (expensive explainers) so the prediction path never blocks; run drift analysis as a cheap hourly/daily offline job.

LIMITATIONS

- **Explainer cost** — cheap for trees, costly for model-agnostic explainers; sample & run out-of-band.
- **Baseline choice** — drift is relative; seasonal traffic needs a seasonally-aware reference.
- **Threshold tuning** — the PSI bands are conventions, not laws; calibrate against a quiet period.
- **Attribution stability** — SHAP is sensitive to correlated features & config; adequate per-window sample sizes and a fixed explainer reduce, but don't eliminate, variance.
- **Leading indicator, not truth** — flags that reasoning changed, not that it's harmful. Triangulate with input drift & performance.

Key refs: Lundberg & Lee, *SHAP* (NeurIPS 2017) · Lundberg et al., *TreeExplainer* (2020) · Siddiqi, *PSI / Credit Risk Scorecards* · adversarial validation · UCI Adult Income.

A third pillar of ML observability. Input drift + delayed-label performance share a blind spot — a model that keeps its accuracy while its reasoning drifts. Explanation drift closes the gap: label-free, leading, and per-feature. **Watch all three.**